

Chapter 3

Describing Syntax and Semantics

Prepared by: Assoc.Prof. Rasha Sakr

3.1 Introduction

3.2 The General Problem of Describing Syntax

3.3 Formal Methods of Describing Syntax

3.4 Attribute Grammars

3.5 Describing the Meanings of Programs: Dynamic Semantics

Summary •

Chapter 3

Describing Syntax and Semantics

3.1 Introduction 110

- **Syntax** – the **form** of the expressions, statements, and program units
- **Semantics** - the **meaning** of the expressions, statements, and program units.
- Ex: the syntax of a Java while statement is

```
while (boolean_expr) statement
```

- The semantics of this statement form is that when the current value of the Boolean expression is true, the embedded statement is executed.
- The form of a statement should strongly suggest what the statement is meant to accomplish.

3.2 The General Problem of Describing Syntax 111

- A **sentence** or “**statement**” is a string of characters over some alphabet. The syntax rules of a language specify which strings of characters from the language’s alphabet are in the language.
- A **language** is a set of sentences.
- A **lexeme** is the lowest level syntactic unit of a language. It includes identifiers, literals, operators, and special word (e.g. *, sum, begin). A **program** is strings of lexemes.
 - A **token** is a category of lexemes (e.g., identifier). An identifier is a token that have lexemes, or instances, such as sum and total.
- Ex:

```
index = 2 * count + 17;
```

<i>Lexemes</i>	<i>Tokens</i>
index	identifier
=	equal_sign
2	int_literal
*	mult_op
count	identifier
+	plus_op
17	int_literal
;	semicolon

Language Recognizers and Generators

- In general, language can be formally defined in two distinct ways: by recognition and by generation.
- **Language Recognizers:**
 - A recognition device reads input strings of the language and decides whether the input strings belong to the language.
 - It only determines whether given programs are in the language.
 - Example: syntax analyzer part of a compiler. The syntax analyzer, also known as **parsers**, determines whether the given programs are syntactically correct.
- **Language Generators:**
 - A device that generates **sentences** of a language
 - One can determine if the syntax of a particular sentence is correct by comparing it to the structure of the generator

3.3 Formal Methods of Describing Syntax 113

- The formal language generation mechanisms are usually called **grammars**
- Grammars are commonly used to describe the **syntax** of programming languages.

3.3.1 Backus-Naur Form and Context-Free Grammars

- It is a syntax description formalism that became the **most widely** used method for programming language syntax.

3.3.1.1 Context-free Grammars

- Developed by Noam Chomsky in the mid-1950s who described four classes of generative devices or grammars that define four classes of languages.
- Context-free and regular grammars are useful for describing the syntax of programming languages.
- Tokens of programming languages can be described by regular grammars.
- Whole programming languages can be described by context-free grammars.

3.3.1.2 Backus-Naur Form (1959)

- Invented by John Backus to describe ALGOL 58 syntax.
- **BNF** (Backus-Naur Form) is equivalent to context-free grammars used for describing syntax.

3.3.1.3 Fundamentals

- A metalanguage is a language used to describe another language. **BNF** is a metalanguage for programming language.
- In BNF, abstractions are used to represent classes of syntactic structures--they act like syntactic variables (also called nonterminal symbols)

`<assign> → <var> = <expression>`

- This is a **rule**; it describes the structure of an assignment statement
- A rule has a left-hand side (**LHS**) “The abstraction being defined” and a right-hand side (**RHS**) “consists of some mixture of tokens, lexemes and references to other abstractions”, and consists of terminal and nonterminal symbols.
- This particular rule specifies that the abstraction `<assign>` is defined as an instance of the abstraction `<var>`, followed by the lexeme `=`, followed by an instance of the abstraction `<expression>`.
- Example:

`total = subtotal1 + subtotal2`

- A grammar is a finite nonempty set of rules and the abstractions are called nonterminal symbols, or simply **nonterminals**.
- The lexemes and tokens of the rules are called terminal symbols or **terminals**.
- A BNF description, or grammar, is simply a **collection of rules**.
- An abstraction (or nonterminal symbol) can have more than one RHS
- For Example, a Java if statement can be described with the rule

`<if_stmt> → if (<logic_expr>) <stmt>
 | if (<logic_expr>) <stmt> else <stmt>`

- Multiple definitions can be written as a single rule, with the different definitions separated by the symbol `|`, meaning logical **OR**.

3.3.1.4 Describing Lists

- Syntactic lists are described using recursion.

`<ident_list> → identifier
 | identifier, <ident_list>`

- A rule is **recursive** if its LHS appears in its RHS.

3.3.1.5 Grammars and derivations

- The sentences of the language are generated through a sequence of applications of the rules, beginning with a special nonterminal of the grammar called the **start symbol**.
- A **derivation** is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols)
- Example 3.1 An Grammar for a Small language:

`<program> → begin <stmt_list> end
<stmt_list> → <stmt> | <stmt> ; <stmt_list>
<stmt> → <var> = <expression>
<var> → A | B | C
<expression> → <var> + <var> | <var> - <var> | <var>`

- An example derivation for a program **begin A = B + C; B = C end**

<program>	=>	begin	<stmt_list>	end
	=>	begin	<stm>; <stmt_list>	end
	=>	begin	<var> = <expression>; <stmt_list>	end
	=>	begin	A = <expression>; <stmt_list>	end
	=>	begin	A = <var> + <var>; <stmt_list>	end
	=>	begin	A = B + <var>; <stmt_list>	end
	=>	begin	A = B + C; <stmt_list>	end
	=>	begin	A = B + C; <stmt>	end
	=>	begin	A = B + C; <var> = <expression>	end
	=>	begin	A = B + C; B = <expression>	end
	=>	begin	A = B + C; B = <var>	end
	=>	begin	A = B + C; B = C	end

- Every string of symbols in the derivation, including <program>, is a sentential form.
- A sentence is a sentential form that has **only** terminal symbols.
- A **leftmost derivation** is one in which the leftmost nonterminal in each sentential form is the one that is expanded. The derivation continues until the sentential form contains no nonterminals.
- A derivation may be neither leftmost nor rightmost.
- Example 3.2 An Grammar for a Small Assignment Statements:

```

<assign>  → <id> = <expr>
<id>      → A | B | C
<expr>    → <id> + <expr>
           | <id> * <expr>
           | (<expr>)
           | <id>

```

- An example derivation for the assignment **A = B * (A + C)**

```

<assign>  => <id> = <expr>
           => A = <expr>
           => A = <id> * <expr>
           => A = B * <expr>
           => A = B * (<expr>)
           => A = B * (<id> + <expr>)
           => A = B * (A + <expr>)
           => A = B * (A + <id>)
           => A = B * (A + C)

```

3.3.1.6 Parse Trees

- Hierarchical structures of the language are called **parse trees**.
- A parse tree for the simple statement **A = B * (A + C)**

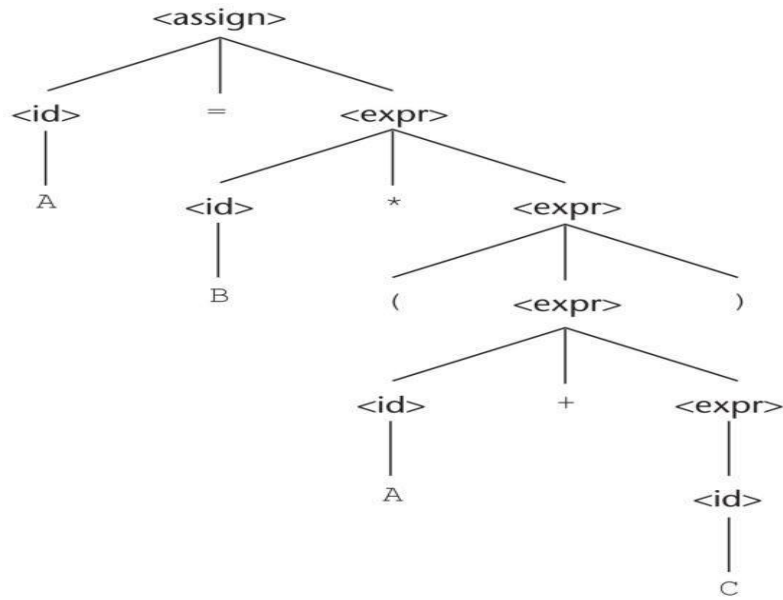


Figure 3.1 A parse tree for the simple statement $A = B * (A + C)$

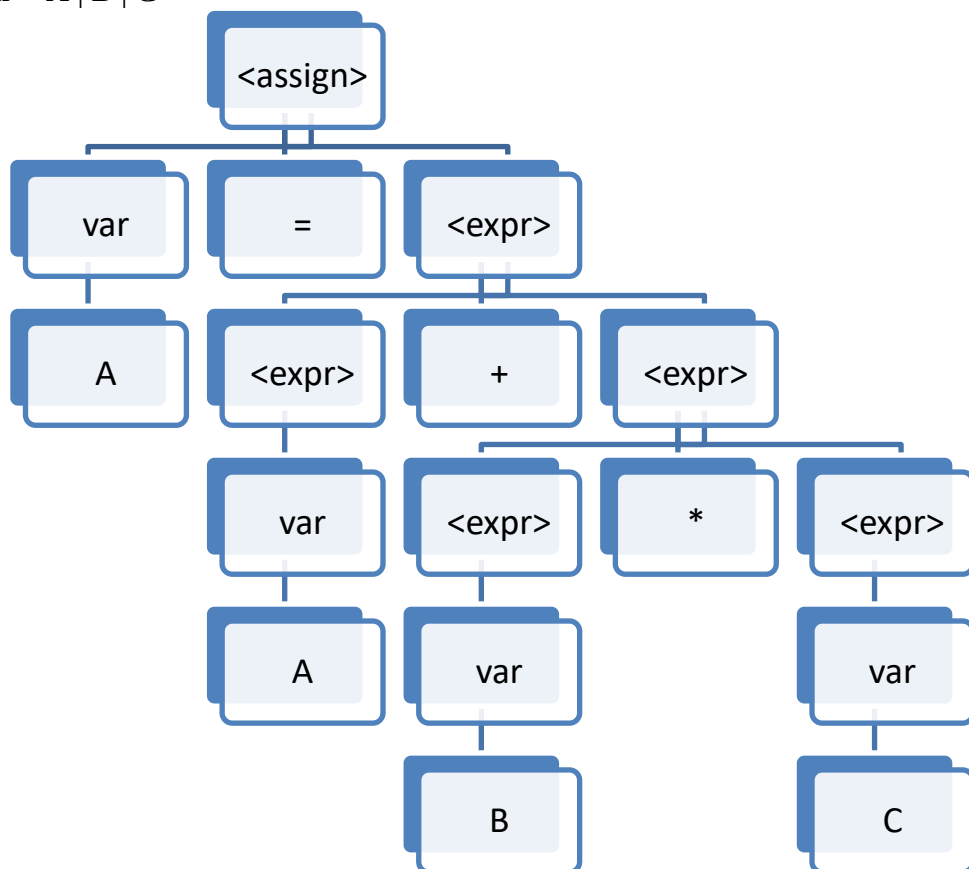
$\langle \text{assign} \rangle ::= \langle \text{var} \rangle \text{'='} \langle \text{expr} \rangle$

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle \text{'+'} \langle \text{expr} \rangle \mid$

$\langle \text{expr} \rangle \text{'*'} \langle \text{expr} \rangle \mid$

$\langle \text{var} \rangle$

$\text{var} = A \mid B \mid C$



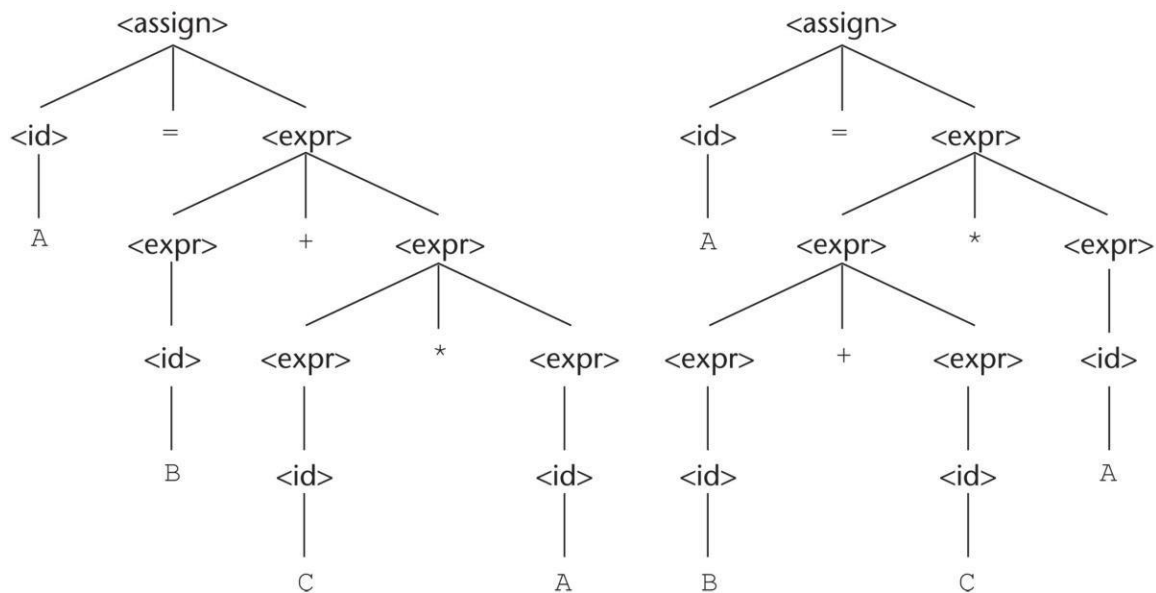
3.3.1.7 Ambiguity

- A grammar is ambiguous if it generates a sentential form that has **two or more** distinct parse trees.
- Example 3.3 An Ambiguous Grammar for Small Assignment Statements
- Two distinct parse trees for the same sentence, **A = B + C * A**

```

<assign>  → <id> = <expr>
<id>      → A | B | C
<expr>    → <expr> + <expr>
           | <expr> * <expr>
           | (<expr>)
           | <id>

```



Copyright ©2016 Pearson Education, All Rights Reserved

Figure 3.2 Two distinct parse trees for the same sentence, **A = B + C * A**

The grammar :

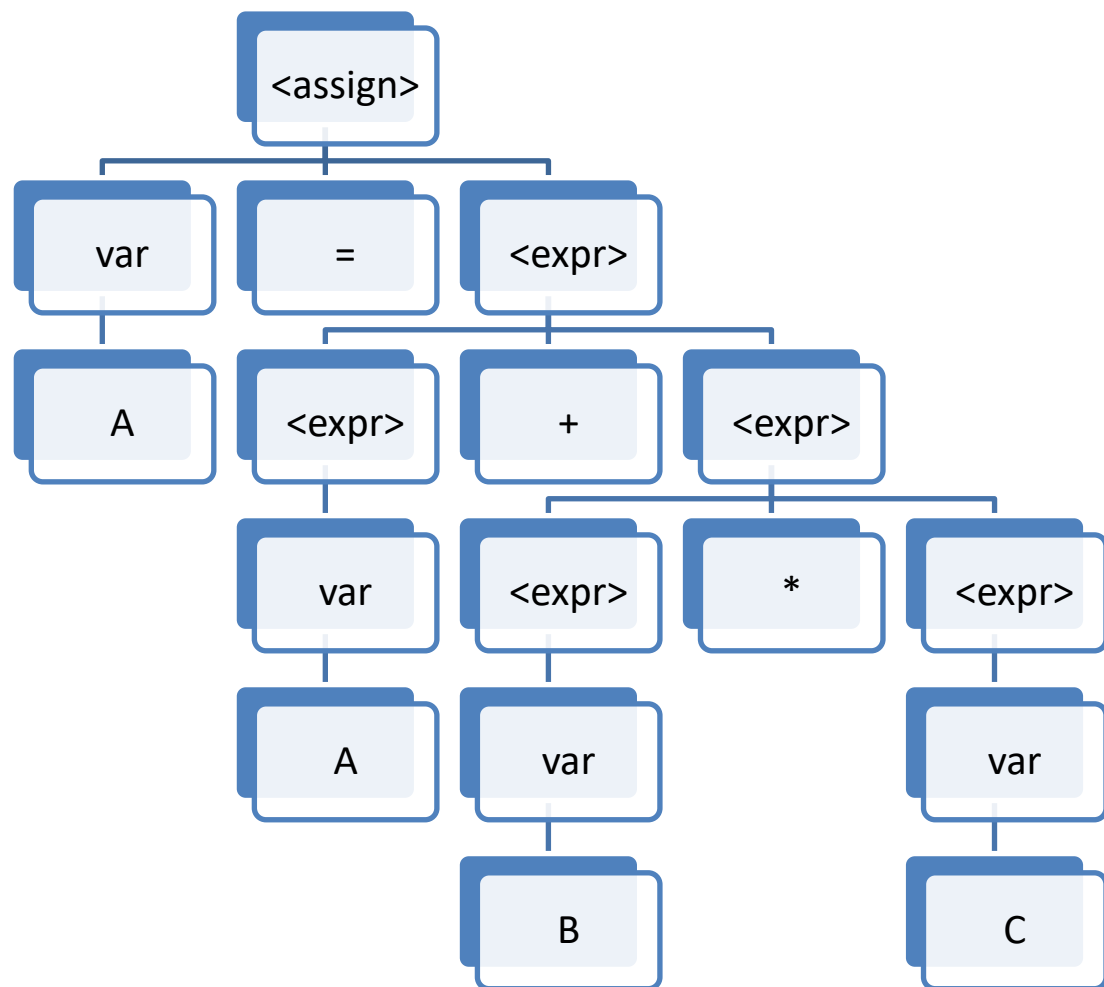
<assign> ::= <var> '=' <expr>

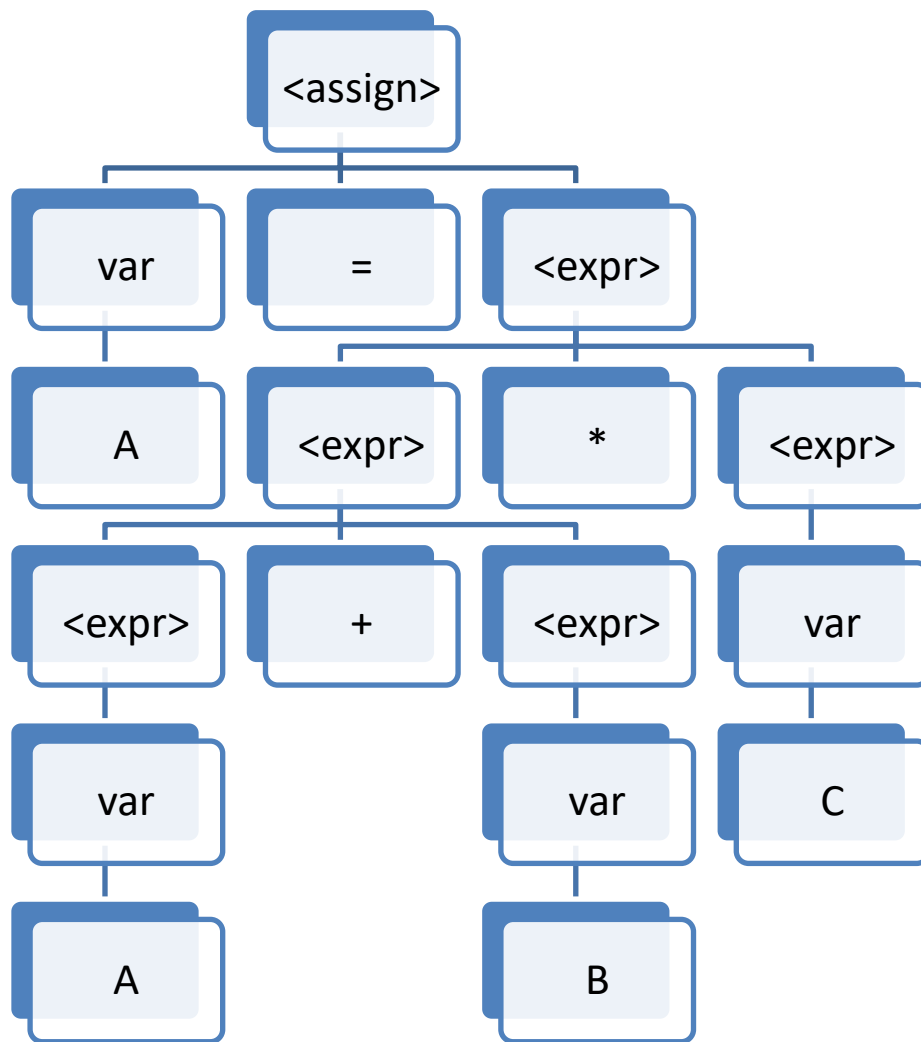
<expr> ::= <expr> '+' <expr> |

<expr> '*' <expr> |

<var>

var = A | B | C





3.3.1.8 Operator Precedence

- The fact that an operator in an arithmetic expression is generated **lower** in the parse tree can be used to indicate that it has **higher precedence** over an operator produced higher up in the tree.
- In the left parsed tree above, one can conclude that the * operator has precedence over the + operator. How about the tree on the right hand side?
- Example 3.4 An **unambiguous** Grammar for Expressions

```

<assign>  → <id> = <expr>
<id>      → A | B | C
<expr>    → <expr> + <term>
           | <term>
<term>    → <term> * <factor>
           | <factor>
<factor>  → (<expr>)
           | <id>
  
```

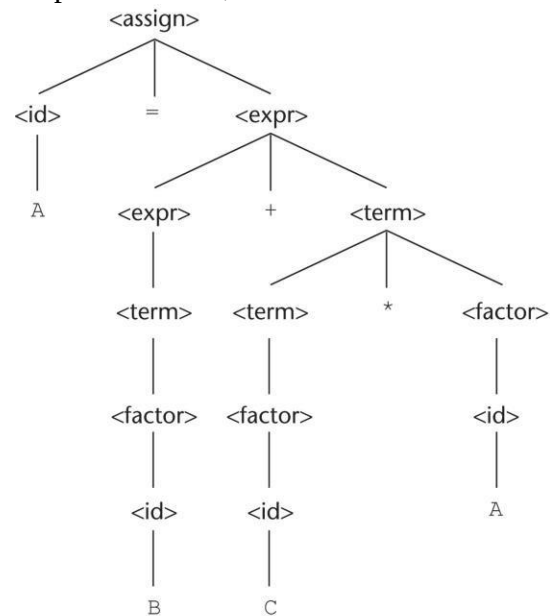
- **Leftmost derivation** of the sentence **A = B + C * A**

```

<assign>  => <id> = <expr>
          => A = <expr>
          => A = <expr> + <term>
          => A = <term> + <term>
          => A = <factor> + <term>
          => A = <id> + <term>
          => A = B + <term>
          => A = B + <term> * <factor>
          => A = B + <factor> * <factor>
          => A = B + <id> * <factor>
          => A = B + C * <factor>
          => A = B + C * <id>
          => A = B + C * A

```

A parse tree for the simple statement, $A = B + C * A$



Copyright © 2016 Pearson Education, All Rights Reserved

Figure 3.3 The unique parse tree for $A = B + C * A$ using an unambiguous grammar

- **Rightmost derivation** of the sentence $A = B + C * A$

```

<assign>  => <id> = <expr>
          => <id> = <expr> + <term>
          => <id> = <expr> + <term> * <factor>
          => <id> = <expr> + <term> * <id>
          => <id> = <expr> + <term> * A
          => <id> = <expr> + <factor> * A
          => <id> = <expr> + <id> * A
          => <id> = <expr> + C * A
          => <id> = <term> + C * A
          => <id> = <factor> + C * A
          => <id> = <id> + C * A
          => <id> = B + C * A
          => A = B + C * A

```

- **Both** of these derivations, however, are represented by the **same** parse tree.

$\langle \text{assign} \rangle ::= \langle \text{id} \rangle \text{'='} \langle \text{expr} \rangle$

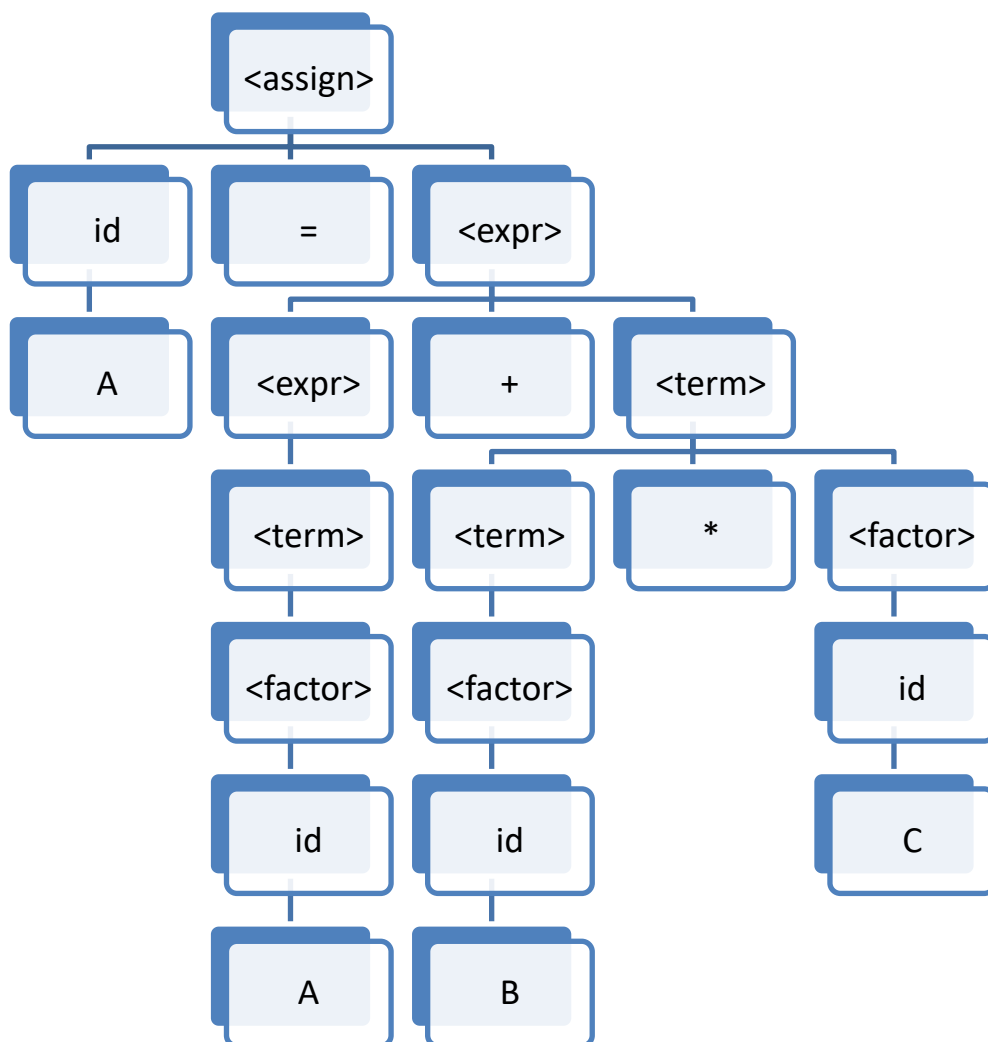
$\text{id} = A \mid B \mid C$

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{term} \rangle \mid$
 $\langle \text{term} \rangle$

$\langle \text{term} \rangle ::= \langle \text{term} \rangle * \langle \text{factor} \rangle \mid$
 $\langle \text{factor} \rangle$

$\langle \text{factor} \rangle ::= \langle \text{expr} \rangle \mid \text{id}$

$A = A + B * C$



3.3.1.9 Associativity of Operators

- Do parse trees for expressions with two or more adjacent occurrences of operators with **equal precedence** have those occurrences in proper hierarchical order?
- An example of an assignment using the previous grammar is: $A = B + C + A$

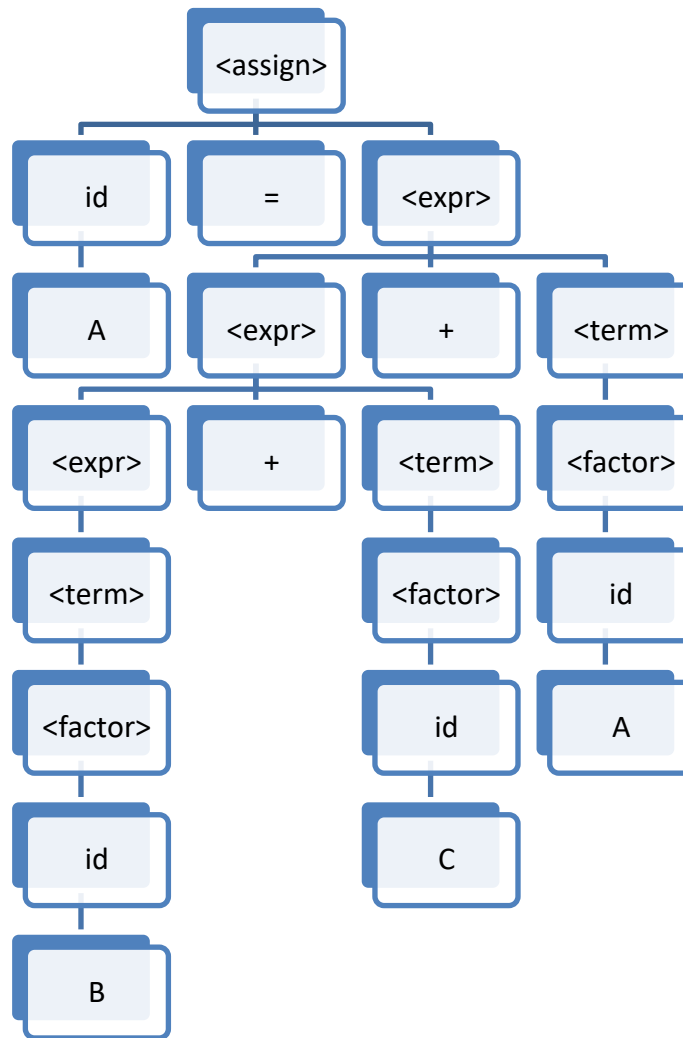


Figure 3.4 A parse tree for $A = B + C + A$ illustrating the associativity of addition

- Figure above shows the left + operator lower than the right + operator. This is the correct order if + operator meant to be **left associative**, which is typical.
- When a grammar rule has LHS also appearing at beginning of its RHS, the rule is said to be left recursive. The left recursion specifies left associativity.
- In most languages that provide it, the **exponentiation operator** is **right associative**. To indicate right associativity, right recursion can be used. A grammar rule is **right recursive** if the LHS appears at the right **end** of the RHS. Rules such as

```

<factor>  → <exp> ** <factor>
          | <exp>
<exp>     → (<exp>)
          | id
  
```

Operator associativity can be indicated by a grammar. For example:

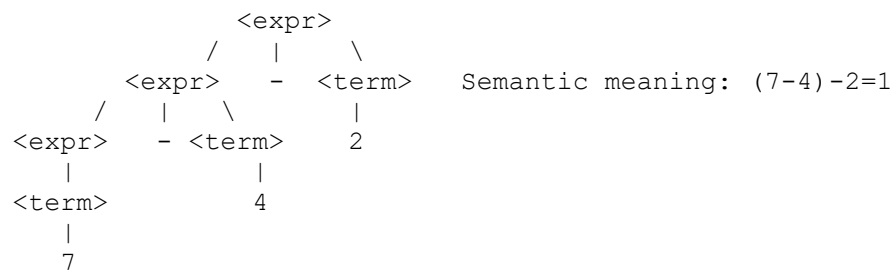
$$7 - 4 - 2$$

Left associativity means the 4 *associates* left: $(7 - 4) - 2 = 1$

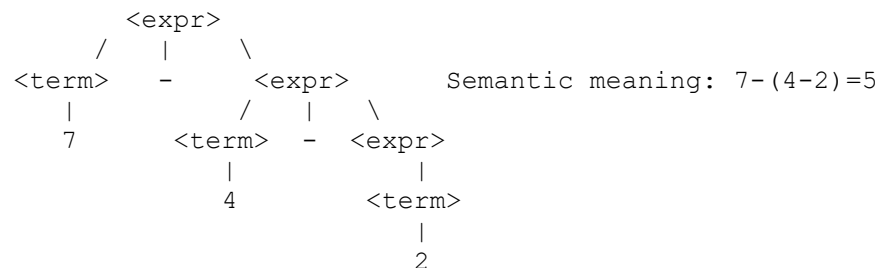
Right associativity means the 4 *associates* right: $7 - (4 - 2) = 5$

Example of recursion in the rule to control associativity with corresponding parse trees for $7 - 4 - 2$.

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle - \langle \text{term} \rangle \mid \langle \text{term} \rangle$ (left recursive)
 $\langle \text{term} \rangle \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$



$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle - \langle \text{expr} \rangle \mid \langle \text{term} \rangle$ (right recursive)
 $\langle \text{term} \rangle \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$



Operator	Description	Associativity
() [] . -> ++ --	Parentheses (function call) (see Note 1) Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement (see Note 2)	left-to-right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of <i>type</i>) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= = <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

3.3.2 Extended BNF

- Because of minor inconveniences in BNF, it has been extended in several ways. EBNF extensions do not enhance the descriptive powers of BNF; they **only** increase its readability and writability.
- Three extensions are commonly included in various versions of EBNF

❖ **Optional** parts are placed in brackets (**[]**)

`<select_stmt> → if ( <expr> ) <stmt> [ else <stmt> ]`

`<if_stmt> → if (expression) <statement> [else <statement>]`

- Without the use of the brackets, the syntactic description of this statement would require the following two rules:

`<if_stmt> → if (expression) <statement>
 | if (expression) <statement> else <statement>`

❖ Put **repetitions** (0 or more) in braces (**{ }**)

`<id_list> → <id> { , <id> }`

`<ident_list> → <identifier> { , <identifier> }`

❖ Put **multiple-choice** (alternative parts) options of RHSs in parentheses and separate them with vertical bars (|, OR operator)

`<term> → <term> ( + | - ) const`

`<term> → <term> ( * | / | % ) <factor>`

- In BNF, a description of this <term> would require the following three rules:

`<term> → <term> * <factor>
 | <term> / <factor>
 | <term> % <factor>`

- Example 3.5 BNF and EBNF Versions of an Expression Grammar

BNF:

`<expr> → <expr> + <term>
 | <expr> - <term>
 | <term>

<term> → <term> * <factor>
 | <term> / <factor>
 | <factor>

<factor> → <exp> ** <factor>
 | <exp>

<exp> → (<expr>)
 | id`

EBNF:

`<expr>       → <term> { ( + | - ) <term> }`

```

<term>      → <factor> { (* | /) <factor> }
<factor>    → <exp> { ** <exp> }
<exp>       → ( <expr> )
              | id

```

Try :

1-Consider the grammar

```

S → AB | AD
A → Aa | b
B → Bb | a
D → cDc | d

```

Are these valid strings in the language generated by the grammar?

ba ?

bbbbab ?

aaaacccc ?

bcdcc ?

baaab ?

baccdcc ?

2- Consider this grammar:

```

<assign> → <id> = <expr>
<id>    → A | B | C
<expr>  → <id> + <expr> | <id> * <expr> | ( <expr> ) | <id>

```

Show a parse tree and a leftmost derivation for the following statement:

A = A * (B + (C * A))

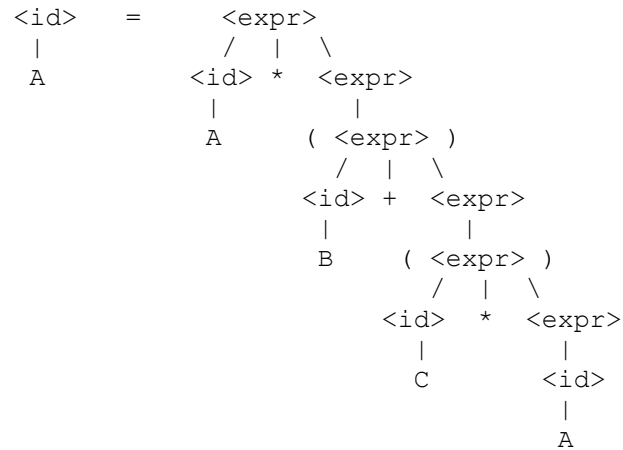
1. <assign> => <id> = <expr>
2. => A = <expr>
3. => A = <id> * <expr>
4. => A = A * <expr>
5. => A = A * (<expr>)
6. => A = A * (<id> + <expr>)
7. => A = A * (B + <expr>)
8. => A = A * (B + (<expr>))
9. => A = A * (B + (<id> * <expr>))
10. => A = A * (B + (C * <expr>))
11. => A = A * (B + (C * <id>))
12. => A = A * (B + (C * A))

Parse tree.

```

      <assign>
      /  |  \

```

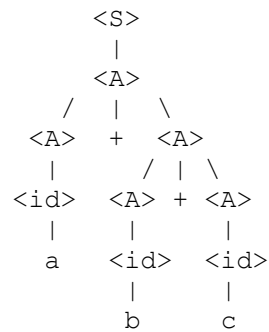



To prove that a grammar is ambiguous, provide a sentence in the grammar that has two different parse trees. (You can't show this with derivations.)

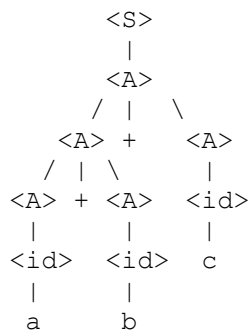
$\langle S \rangle \rightarrow \langle A \rangle$
 $\langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle \mid \langle \text{id} \rangle$
 $\langle \text{id} \rangle \rightarrow a \mid b \mid c$

Sentence: $a + b + c$

Tree 1.



Tree 2.



Describe the language defined by the following grammar:

$\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b$
 $\langle A \rangle \rightarrow \langle A \rangle b \mid b$
 $\langle B \rangle \rightarrow a \langle B \rangle \mid a$

Start by finding the behavior of $\langle A \rangle$ and $\langle B \rangle$:

$\langle A \rangle \rightarrow b$

```

        bb
        bbb ...

<A> is 1 or more "b"s.

<B> -> a
        aa
        aaa ...

<B> is 1 or more "a"s

Thus,

<A>a<B>b defines the language of all strings of 1 or more "b"s followed
by 2 or more "a"s ending with a "b".

```

3.4 Attribute Grammars

- An attribute grammar is a device used to describe **more** of the structure of a programming language than can be described with a context-free grammar.
- An attribute grammar is a generative grammar that can describe both the context-free and the context-sensitive syntax of a language. Attribute grammars add context-sensitive information to CFGs by transporting semantic information in a controlled fashion up and down the parse tree. Information is passed through synthesized and inherited attributes that are attached to the nodes of the parse tree.

3.4.1 Static Semantics

Static Semantics refers to the context-sensitive information in a programming language that can be determined prior to runtime. Normally, semantics describes the behavior of a program during execution but in this context reflects the context-sensitive meaning of a program. Examples of static semantics include type checking and the location of declarations. BNF/Context-free grammars (CFGs) are well-suited to describe the context-free syntax of programming languages but not easily context-sensitive information without becoming large & clumsy. Since the size of a grammar determines the size of the parser, attribute grammars enhance compilers with static semantics checking.

- Context-free grammars (CFGs) cannot describe all of the syntax of programming languages.
- In Java, for example, a floating-point value **cannot** be assigned to an integer type variable, although the opposite is legal.
- The **static semantics** of a language is only indirectly related to the meaning of programs during execution; rather, it has to do with the legal forms of programs (syntax rather than semantics).
- Many static semantic rules of a language state its type constraints. Static semantics is so named because the analysis required to these specifications can be done at **compile time**.
- **Attribute grammars** was designed by Knuth (1968) to describe both the **syntax** and the

static semantics of programs.

3.4.2 Basic Concepts

- Attribute grammars have additions to are context-free grammars to carry some **semantic** information on parse tree nodes.
- Attribute grammars are context-free grammars to which have been added attributes, attribute computation functions, and predicate function.

3.4.3 Attribute Grammars Defined

- Associated with each grammar symbol X is a set of attributes $A(X)$.
 - The set $A(X)$ consists of two disjoint set $S(X)$ and $I(X)$, call **synthesized and inherited attributes**.
 - **Synthesized attributes** are used to pass semantic information **up** a parse tree, while **inherited attributes** pass semantic information **down** and across tree.
- Let $X_0 \rightarrow X_1 \dots X_n$ be a rule
 - Functions of the form $S(X_0) = f(A(X_1), \dots, A(X_n))$ define synthesized attributes
 - Functions of the form $I(X_j) = f(A(X_0), \dots, A(X_n))$, for $i \leq j \leq n$, define inherited attributes
 - Initially, there are **intrinsic attributes** on the leaves
- **Synthesized attributes** pass information from child to parent up the tree. The parse tree is "decorated" with synthesized attributes in a bottom-up fashion. If the value of a synthesized attribute on a leaf node is assigned before parsing begins (hence outside the parse tree), the attribute is called intrinsic.
- **Intrinsic attributes** are synthesized attributes of leaf nodes, the value of which is known before the sentence is parsed. These attributes pass external information (such as from the symbol table) up the parse tree.
- **Inherited attributes** pass information from parent to child and from sibling to sibling. The overall effect is to move information down the parse tree. The parse tree is "decorated" in a top-down fashion for inherited attributes. Each grammar symbol may have inherited or synthesized attributes, or both.
- Note: If the tree contains both inherited and synthesized attributes, a combination of top-down and bottom-up order is used to decorate the tree.

3.4.4 Intrinsic Attributes

- Intrinsic attributes are **synthesized** attributes of **leaf** nodes whose values are determined **outside** the parse tree.
- For example, the type of an instance of a variable in a program could come from the **symbol table**, which is used to store variable names and their types.

As a very simple example of how attribute grammars can be used to describe static semantics, consider the following fragment of an attribute grammar that describes the rule that the name on the end of an Ada procedure must match the procedure's name. (This rule cannot be stated in BNF.) The string attribute of `<proc_name>`, denoted by `<proc_name>.string`, is the actual string of characters that were found immediately following the reserved word `procedure` by the compiler. Notice that when there is more than one occurrence of a nonterminal in a syntax rule in an attribute grammar, the nonterminals are subscripted with brackets to distinguish them. Neither the subscripts nor the brackets are part of the described language.

In this example, the [predicate rule](#) states that the name string attribute of the `<proc_name>` nonterminal in the subprogram header must match the name string attribute of the `<proc_name>` nonterminal following the end of the subprogram. Next, we consider a larger example of an attribute grammar. In this case, the example illustrates how an attribute grammar can be used to check the type rules of a simple assignment statement. The syntax and static semantics of this assignment statement are as follows: The only variable names are A, B, and C. The right side of the assignments can be either a variable or an expression in the form of a variable added to another variable. The variables can be one of two types: int or real. When there are two variables on the right side of an assignment, they need not be the same type. The type of the expression when the operand types are not the same is always real. When they are the same, the expression type is that of the operands. The type of the left side of the assignment must match the type of the right side. So the types of operands in the right side can be mixed, but the assignment is valid only if the target and the value resulting from evaluating the right side have the same type. The attribute grammar specifies these static semantic rules.

1. Syntax rule: $\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$
Semantic rule: $\langle \text{expr} \rangle.\text{expected_type} \leftarrow \langle \text{var} \rangle.\text{actual_type}$
2. Syntax rule: $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle[2] + \langle \text{var} \rangle[3]$
Semantic rule: $\langle \text{expr} \rangle.\text{actual_type} \leftarrow$
if ($\langle \text{var} \rangle[2].\text{actual_type} = \text{int}$ and ($\langle \text{var} \rangle[2].\text{actual_type} = \text{int}$)
then int
else real
end if
Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$
3. Syntax rule: $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle$
Semantic rule: $\langle \text{expr} \rangle.\text{actual_type} \leftarrow \langle \text{var} \rangle.\text{actual_type}$
Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$
4. Syntax rule: $\langle \text{var} \rangle \rightarrow A \mid B \mid C$

Semantic rule: $\langle \text{var} \rangle.\text{actual_type} \leftarrow \text{look-up}(\langle \text{var} \rangle.\text{string})$

The look-up function looks up a given variable name in the symbol table and returns the variable's type

- The syntax portion of our example attribute grammar is

```

<assign> → <var> = <expr>
<expr>  → <var>[2] + <var>[3]
          | <var>
<var>    → A | B | C
  
```

- actual_type - A **synthesized** attribute associated with nonterminal <var> and <expr>.
 - It is used to store the actual type, int or real, or a variable or expression.
 - In the case of a variable, the actual type is intrinsic.
 - In the of an expression, it is determined from the actual types of child node or children nodes of the <expr> nonterminal.
- expected_type - A **inherited** attribute associated with nonterminal <expr>.
 - It is used to store the type, either int or real, that is expected for the expression, as determined by the type of the variable on the left side of the assignment statement.

3.4.6 Computing Attribute Values

- Now, consider the process of computing the attribute values of a parse tree, which is sometimes called decorating the parse tree.
- The tree in Figure 3.7 show the flow of attribute values in the example of Figure 3.6.

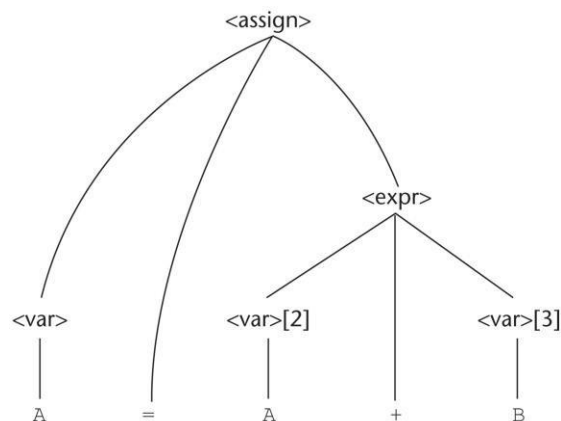


Figure 3.6 A parse tree for $A = A + B$

- The following is an evaluation of the attributes, in an order in which it is possible to computer them:

1. $\langle \text{var} \rangle.\text{actual_type}$	$\leftarrow$	look-up (A)	(Rule 4)
2. $\langle \text{expr} \rangle.\text{expected_type}$	$\leftarrow$	$\langle \text{var} \rangle.\text{actual_type}$	(Rule 1)

3. <var>[2].actual_type	←	look-up (A)	(Rule 4)
<var>[3].actual_type	←	look-up (B)	(Rule 4)
4. <expr>.actual_type	←	either int or real	(Rule 2)

5. <expr>.expected_type == <expr>.actual_type is either TRUE or FALSE (Rule 2)

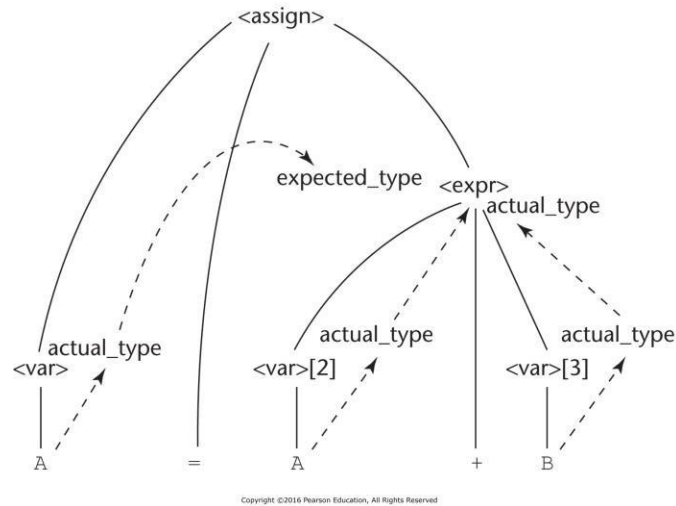


Figure 3.7 The flow of attributes in the tree

- The tree in Figure 3.8 shows the final attribute values on the nodes. In this example, A is defined as a real and B is defined as an int.

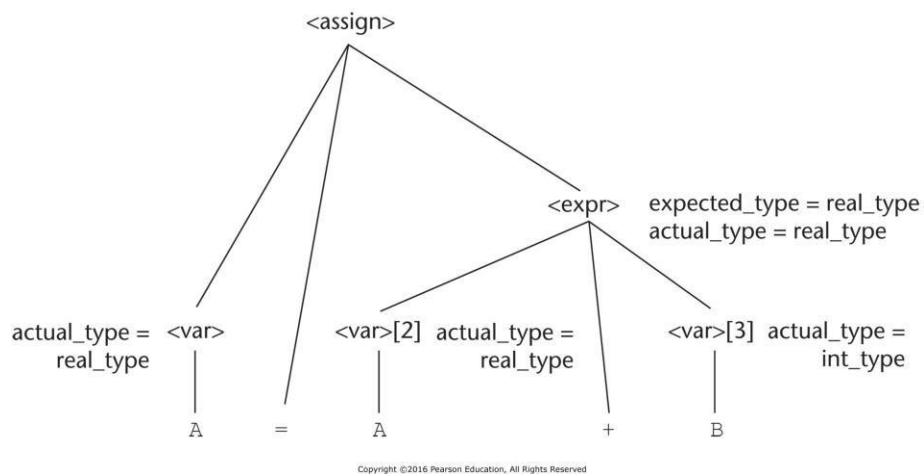
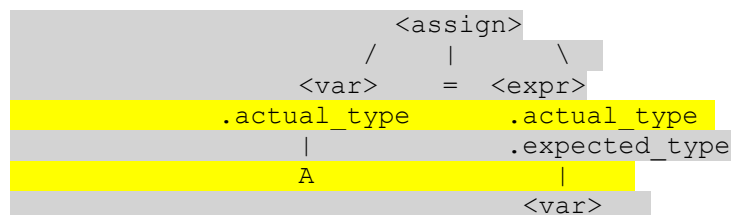


Figure 3.8 A fully attributed parse tree (A: real, B: int)

The parse tree for A = C



.actual_type

|

C

Assume:

int A, B; real C

A = B + C

B = C

What happens?

* How are attribute values computed?

Initially, there are only intrinsic attributes on the leaves...

1. `<var>.actual_type <- look-up(A)` (Rule 4)
2. `<expr>.expected_type <- <var>.actual_type` (Rule 1)
3. `<var>[2].actual_type <- lookup(A)` (Rule 4)
`<var>[3].actual_type <- lookup(B)` (Rule 4)
4. `<var>[1].actual_type <- either int or real` (Rule 2)
5. `<expr>.expected_type == <expr>.actual_type` is either TRUE or FALSE

3.4.7 Evaluation

- Checking the static semantic rules of a language is an essential part of all **compiler**.
 - One of the main difficulties in using an attribute grammar to describe all of the syntax and static semantics of a real contemporary programming language is the **size and complexity** of the attribute grammar.
 - Furthermore, the attribute values on a large parse tree are **costly** to evaluate.

3.5 Describing the Meanings of Programs: Dynamic Semantics 134

- Three methods of semantic description:
 - **Operational semantics:** It is a method of **describing** the meaning of language constructs in terms of their effects on an ideal machine.
 - **Denotation semantics:** **Mathematical** objects are used to represent the meanings of language constructs. Language entities are converted to these mathematical objects with recursive functions.
 - **Axiomatic semantics:** It is based on formal **logic** and devised as a tool for proving the correctness of programs.

❖ Denotational vs Operational Semantics

- In operational semantics, the **state changes** are defined by **coded algorithms**
- In denotational semantics, the **state changes** are defined by rigorous **mathematical functions**

3.5.1 Operational Semantics

- The idea behind **operational semantics** is to describe the meaning of a statement or program, by specifying the effects of running it on a machine. The effects on the machine are viewed as the sequence of **changes in its states**, where the machine's state is the collection of the collection of the values in its storage.
- Most programmers have written a small **test** program to determine the meaning of some programming language construct.
- The basic process of operational semantics is not unusual. In fact, the concept is frequently used in programming textbooks and programming in reference manuals.
- For example, the semantics of the C **for** construct can be described in terms of simpler statements, as in

<i>C Statement</i>	<i>Meaning</i>
for (expr1; expr2; expr3) {	expr1;
...	loop: if expr2 == 0 goto out
}	...
	expr3;
	goto loop
	out: ...

- Operational semantics **depends** on programming languages of lower level, **not** mathematics and logic.

3.5.2 Denotational Semantics

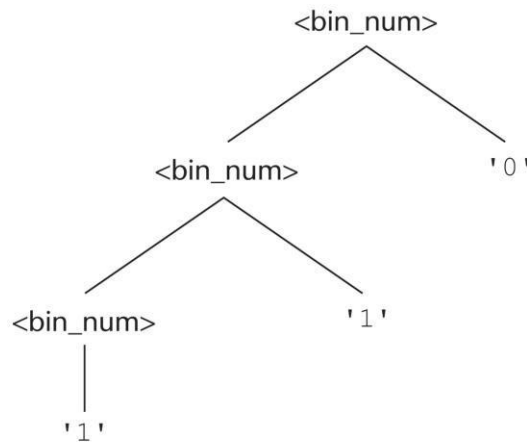
- **Denotational semantics** is the most rigorous and most widely known formal method for describing the meaning of programs.
- It is solidly based on **recursive** function theory.

Two Simple Examples

- Example 1:
 - We use a very simple language construct, character string representations of **binary numbers**, to introduce the denotational method.
 - The syntax of such binary numbers can be described by the following grammar rules:

```
<bin_num>  → '0'
            | '1'
            | <bin_num> '0'
            | <bin_num> '1'
```

- A parse tree for the example binary number, **110**, is show in Figure 3.9.



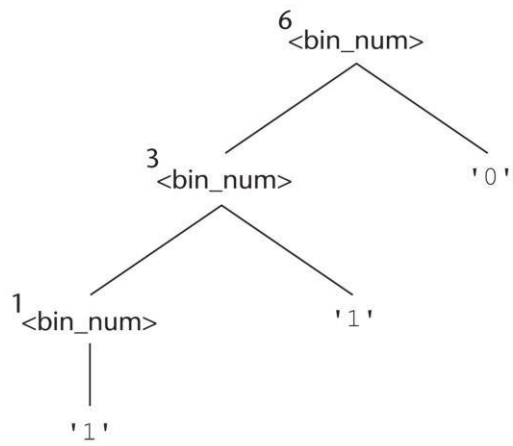
Copyright ©2016 Pearson Education, All Rights Reserved

Figure 3.9 A parse tree of the binary number 110

- The semantic function, named M_{bin} , maps the syntactic objects, as described in the previous grammar rules, to the objects in N , the set of non-negative decimal numbers. The function M_{bin} is defined as follows:

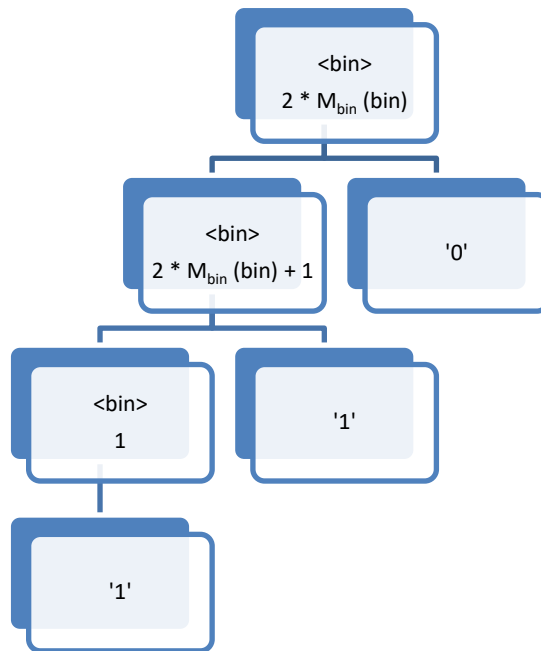
```
 $M_{\text{bin}}('0') = 0$ 
 $M_{\text{bin}}('1') = 1$ 
 $M_{\text{bin}}(<\text{bin\_num}> '0') = 2 * M_{\text{bin}}(<\text{bin\_num}>)$ 
 $M_{\text{bin}}(<\text{bin\_num}> '1') = 2 * M_{\text{bin}}(<\text{bin\_num}>) + 1$ 
```

- The meanings, or denoted objects (which in this case are decimal numbers), can be attached to the nodes of the parse tree, yielding the tree in Figure 3.10)



Copyright ©2016 Pearson Education, All Rights Reserved

Figure 3.10 A parse tree with denoted objects for 110



- Example 2:

Decimal Numbers

- The following denotational semantics description maps decimal numbers as strings of symbols into numeric values
 - The syntactic domain is the set of character string representations of **decimal numbers**. The semantic domain is once again the set N.
 - **Syntax rule:**

$$\begin{aligned} \langle \text{dec_num} \rangle &\rightarrow '0' \mid '1' \mid '2' \mid '3' \mid '4' \mid '5' \mid '6' \mid '7' \mid '8' \mid '9' \\ &\mid \langle \text{dec_num} \rangle ('0' \mid '1' \mid '2' \mid '3' \mid '4' \mid '5' \mid '6' \mid '7' \mid '8' \mid '9') \end{aligned}$$
 - The **Denotational Semantics** mapping for these syntax rules are

$$M_{\text{dec}}('0') = 0, M_{\text{dec}}('1') = 1, M_{\text{dec}}('2') = 2, \dots, M_{\text{dec}}('9') = 9$$

$$M_{\text{dec}}(\langle \text{dec_num} \rangle '0') = 10 * M_{\text{dec}}(\langle \text{dec_num} \rangle)$$

$$M_{\text{dec}}(\langle \text{dec_num} \rangle '1') = 10 * M_{\text{dec}}(\langle \text{dec_num} \rangle) + 1$$

...

$$M_{\text{dec}}(\langle \text{dec_num} \rangle '9') = 10 * M_{\text{dec}}(\langle \text{dec_num} \rangle) + 9$$

Note: M_{dec} is a **semantic function** that maps syntactic objects to a set of non-negative decimal integer values

3.5.3 Axiomatic Semantics

- Axiomatic Semantics is based on **mathematical logic**.
- It is defined in conjunction with the development of a method to **prove** the correctness of programs.
 - Such correction proofs, when they can be constructed, show that a program performs the

computation described by its specification.

- In a proof, each statement of a program is both preceded and followed by a logical expression that specified constraints on program variables.
- Approach: Define axioms or inference rules for each statement type in the language (to allow transformations of expressions to other expressions.)
 - The expressions are called **assertions**.

3.5.3.1 Assertions

- The logical expressions are called predicates, or **assertions**.
- An assertion before a statement (a **precondition**) states the relationships and constraints among variables that are true at that point in execution.
- An assertion following a statement is a **postcondition**.

3.5.3.2 Weakest Preconditions

- A **weakest precondition** is the least restrictive precondition that will guarantee the validity of the associated postcondition.
- The usual notation for specify the axiomatic semantic of a given statement form is

$\{P\} \text{ statement } \{Q\}$

where P is the precondition, Q is the postcondition, and S is the statement form

- An example: $a = b + 1 \quad \{a > 1\}$

One possible precondition: $\{b > 10\}$

Weakest precondition: **$\{b > 0\}$**

- If the weakest precondition can be computed from the given postcondition for each statement of a language, then correctness proofs can be constructed from programs in that language.
- **Program proof process:** The postcondition for the whole program is the desired result. Work back through the program to the first statement. If the precondition on the first statement is the same as the program spec, the program is correct.
- An **Axiom** is a logical statement that is assumed to be true.

- An **Inference Rule** is a method of inferring the truth of one assertion on the basis of the values of other assertions.

$$\frac{S1, S2, \dots, Sn}{S}$$

- The rule states that if S1, S2, ..., and Sn are true, then the truth of S can be inferred. The top part of an inference rule is called its antecedent; the bottom part is called its consequent.

3.5.3.3 Assignment Statements

Assignment Command The first command we examine is assignment, beginning with three examples of preconditions and postconditions for assignment commands:

Example 1: $\{ k = 5 \} k := k + 1 \{ k = 6 \}$

Example 2: $\{ j = 3 \text{ and } k = 4 \} j := j + k \{ j = 7 \text{ and } k = 4 \}$

Example 3: $\{ a > 0 \} a := a - 1 \{ a \geq 0 \}$.

For these simple examples, correctness is easy to prove either proceeding from the precondition to the postcondition or from the postcondition to the precondition. However, many times starting with the postcondition and working backward to derive the precondition proves easier (at least initially). We assume expressions with no side effects in the assignment commands, so only the target variable is changed. “Working backward” means substituting the expression on the right-hand side of the assignment for every occurrence of the target variable in the postcondition and deriving the precondition, following the principle that whatever is true about the target variable after the assignment must be true about the expression before the assignment. Consider the following examples:

Example 1

$\{ k = 6 \}$ postcondition
 $\{ k + 1 = 6 \}$ substituting $k + 1$ for k in postcondition
 $\{ k = 5 \}$ precondition, after simplification.

Example 2

$\{ j = 7 \text{ and } k = 4 \}$ postcondition
 $\{ j + k = 7 \text{ and } k = 4 \}$ substituting $j + k$ for j in postcondition
 $\{ j = 3 \text{ and } k = 4 \}$ precondition, after simplification.

Example 3

$\{ a \geq 0 \}$ postcondition
 $\{ a - 1 \geq 0 \}$ substituting $a - 1$ for a in postcondition $\{ a \geq 1 \}$ simplification
 $\{ a > 0 \}$ precondition, since $a \geq 1 \equiv a > 0$ assuming a is an integer

- Ex:

$$a = b / 2 - 1 \{ a < 10 \}$$

The weakest precondition is computed by substituting $b / 2 - 1$ in the assertion $\{ a < 10 \}$ as follows:

$$\begin{array}{ll} b / 2 - 1 < 10 \\ b / 2 & < 11 \\ b & < 22 \end{array}$$

∴ the weakest precondition for the given assignment and the postcondition is **{b < 22}**

- An assignment statement has a side effect if it changes some variable other than its left side.
- Ex:

$$x = 2 * y - 3 \quad \{x > 25\}$$

The weakest precondition is computed as follows:

$$\begin{array}{rcl} 2 * y - 3 & > & 25 \\ 2 * y & > & 28 \\ y & > & 14 \end{array}$$

∴ the weakest precondition for the given assignment and the postcondition is **{y > 14}**

- Ex:

$$x = x + y - 3 \quad \{x > 10\}$$

The weakest precondition is computed as follows:

$$\begin{array}{rcl} x + y - 3 & > & 10 \\ y & > & 13 - x \end{array}$$

∴ the weakest precondition for the given assignment and the postcondition is **{y > 13 - x}**

3.5.3.4 Sequences

- The weakest precondition for a sequence of statements cannot be described by an axiom, because the precondition depends on the particular kinds of statements in the sequence.
- In this case, the precondition can only be described with an inference rule.
- Let S1 and S2 be adjacent program statements. If S1 and S2 have the following preconditions and postconditions.

$$\begin{array}{l} \{P1\} \quad S1 \quad \{P2\} \\ \{P2\} \quad S2 \quad \{P3\} \end{array}$$

The inference rule for such a two-statement sequence is

$$\frac{\{P1\} \quad S1 \quad \{P2\}, \quad \{P2\} \quad S2 \quad \{P3\}}{\{P1\} \quad S1, \quad S2 \quad \{P3\}}$$

- Ex:

```

y = 3 * x + 1;
x = y + 3;      {x < 10}

```

The weakest precondition is computed as follows:

```

y + 3 < 10
y      < 7

3 * x + 1 < 7
3 * x      < 6
x          < 2

```

∴ the weakest precondition for the first assignment statement is **{x < 2}**

3.5.3.5 Selection

- We next consider the inference rule for selection statements, the general form of which is

```

if B then S1 else S2

```

We consider only selections that include else clause. The inference rule is

$$\frac{\{B \text{ and } P\} S1 \{Q\}, \{\text{not } B \text{ and } P\} S2 \{Q\}}{\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}}$$

- Example of selection statement is

```

If (x > 0) then
    y = y - 1;
else
    y = y + 1;
{y > 0}

```

We can use the axiom for assignment on the then clause

$y = y - 1 \{y > 0\}$

This produce precondition $\{y - 1 > 0\}$ or **{y > 1}**

Now we apply the same axiom to the else clause

$y = y + 1 \{y > 0\}$

This produce precondition $\{y + 1 > 0\}$ or **{y > -1}**

$y > 1 \text{ AND } y > -1$

{y > 1}

Because $\{y > 1\} \Rightarrow \{y > -1\}$, the rule of consequence allows us to use **{y > 1}** for the precondition of selection statement.

3.5.3.6 Logical Pretest Loops

- Computing the weakest precondition (wp) for a while loop is inherently more difficult than for a sequence b/c the number of iterations cannot always be predetermined.
- The corresponding step in the axiomatic semantics of a **while** loop is finding an assertion called a **loop invariant**, which is crucial to finding the weakest precondition.
- The inference rule for computing the precondition for a while loop is as follows:

$$\frac{\{I \text{ and } B\} S \{I\}}{\{I\} \text{ while } B \text{ do } S \text{ end } \{I \text{ and } (\text{not } B)\}}$$

In this rule, I is the loop invariant

- The axiomatic description of a while loop written as

`{P} while B do S end {Q}`

- It is helpful to treat the process of producing the wp as a function, **wp**.

`wp(statement, postcondition) = precondition`

- To find **I**, we use the loop postcondition to compute preconditions for several different numbers of iterations of the loop body, starting with none. If the loop body contains a single assignment statement, the axiom for assignment statements can be used to compute these cases.
- Characteristics of the loop invariant: **I** must meet the following conditions:
 - `P => I` -- the loop invariant must be initially true
 - `{I} B {I}` -- evaluation of the Boolean must not change the validity of I
 - `{I and B} S {I}` -- I is not changed by executing the body of the loop
(Each execution of the loop preserves the invariant)
 - `(I and (not B)) => Q` -- if I is true and B is false, Q is implied
(The invariant and the loop exit condition imply the postcondition)
 - The loop terminates -- can be difficult to prove

Proof of implied : $(P \rightarrow I)$

Proof of implied : $((I \wedge (\neg B)) \rightarrow Q)$

I is called a **loop invariant**. We need to determine I!

What is a loop invariant?

A **loop invariant** is:

- ▶ A relationship among the variables. (A predicate formula involving the variables.)
- ▶ The word “invariant” means something that does not change.
- ▶ It is true before the loop begins.
- ▶ It is true at the start of every iteration of the loop and at the end of every iteration of the loop.
- ▶ It is true after the loop ends.

- Ex:

while $y < x$ **do** $y = y + 1$ **end** $\{y = x\}$

For 0 iterations, the wp is $\rightarrow \{y = x\}$

For 1 iteration,

$\text{wp}(y = y + 1, \{y = x\}) = \{y + 1 = x\}$, or $\{y = x - 1\}$

For 2 iterations,

$\text{wp}(y = y + 1, \{y = x - 1\}) = \{y + 1 = x - 1\}$, or $\{y = x - 2\}$

For 3 iterations,

$\text{wp}(y = y + 1, \{y = x - 2\}) = \{y + 1 = x - 2\}$, or $\{y = x - 3\}$

It is now obvious that $\{y < x\}$ will suffice for cases of one or more iterations. Combining this with $\{y = x\}$ for the 0 iterations case, we get **$\{y \leq x\}$** which can be used for the **loop invariant**. This is the first condition (Proof of implied : $(P \rightarrow I)$)

- We need to prove the second condition $\{I \text{ and } B\} S \{I\} == \{\text{Precondition}\} S \{\text{Postcondition}\}$
- We will get the **Precondition (P)**

$$y = y + 1 \{y \leq x\}$$

$$\{y + 1 \leq x\}$$

$$\{y \leq x - 1\}$$

Precondition is $\{y < x\} \rightarrow 1$
- We will get $\{I \text{ and } B\} \dots \dots \dots \{y \leq x \text{ AND } y < x\} = \{y < x\} \rightarrow 2$
- From 1 and 2 we prove the second condition $\{I \text{ and } B\} S \{I\}$
- The third condition $\{I \text{ and (not } B)\} \rightarrow Q$

$$\{I \text{ and (not } B)\}$$

$$\{y \leq x \text{ AND (not } y < x)\}$$

$$\{y \leq x \text{ AND } y = x\}$$

$$\{y = x\} \{ \text{Postcondition} \} \rightarrow 3$$

while $y < x$ **do** $y = y + 1$ **end** **$\{y = x\}$ means that $Q \{y = x\} \rightarrow 4$**

From 3, 4 we prove the third condition
- The last condition is **The termination of Loop** which is difficult to prove but when we reached **$\{y = x\}$** , the loop will terminate because the truth value of condition $y < x$ is False

Ex:

while $s > 1$ **do** $s = s / 2$ **end** $\{s = 1\}$

For 0 iterations, the wp is $\rightarrow \{s = 1\}$

For 1 iteration,

$\text{wp}(s > 1, \{s = s / 2\}) = \{s / 2 = 1\}$, or $\{s = 2\}$

For 2 iterations,

$\text{wp}(s > 1, \{s = s / 2\}) = \{s / 2 = 2\}$, or $\{s = 4\}$

For 3 iterations,

$\text{wp}(s > 1, \{s = s / 2\}) = \{s / 2 = 4\}$, or $\{s = 8\}$

- Loop Invariant I is $\{s \text{ is a nonnegative power of } 2\}$
- The loop invariant **I** is a weakened version of the loop postcondition, and it is also a

precondition.

- **I** must be weak enough to be satisfied prior to the beginning of the loop, but when combined with the loop exit condition, it must be strong enough to force the truth of the postcondition.

The structure of while rule in the figure (11.2) uses Loop Invariant(P not I) And C instead of S

While Command and Loop Invariants

Continuing the axiomatic definition of Wren, we specify the **while** command:

$$\frac{\{P \text{ and } B\} C \{P\}}{\{P\} \text{ while } B \text{ do } C \text{ end while } \{P \text{ and } (\text{not } B)\}} \quad (\text{While})$$

In this definition **P** is called the **loop invariant**. This assertion captures the essence of the **while** loop: It must be true initially, it must be preserved after the loop body executes, and, combined with the exit condition, it implies the assertion that follows the loop. Figure 11.2 illustrates the situation.

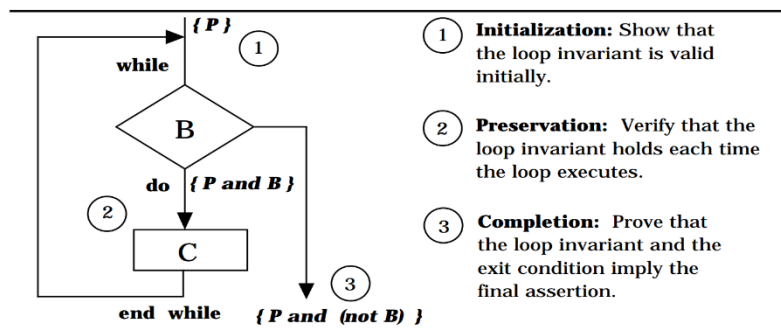


Figure 11.2: Structure of the While Rule

Summary

- **BNF** and context-free grammars are equivalent meta-languages
 - Well-suited for describing the **syntax** of programming languages
- An **attribute grammar** is a descriptive formalism that can describe both the syntax **and** the semantics of a language
- Three primary methods of semantics description
 - **Operation, denotational, axiomatic**

TRY :

- 1- Grammar that produce more than one Parse tree for same sentence is _____
 - a) Ambiguous
 - b) Unambiguous
 - c) Complementation

d) Concatenation Intersection

2- Which of these does not belong to CFG?

- a) Terminal Symbol
- b) Non terminal Symbol
- c) Start symbol
- d) End Symbol

3- If the lexical analyser finds a lexeme with the same name as that of a reserved word, it

-
- a) overwrites the word
 - b) overwrites the functionality
 - c) generates an error
 - d) something else

4- Lexemes can be referred to as:

- a) elements of lexicography
- b) sequence of alphanumeric characters in a token
- c) lexical errors
- d) none of the mentioned

5- Given Grammar: $S \rightarrow A$, $A \rightarrow aA$, $A \rightarrow e$, $B \rightarrow bA$

Which among the following productions are Useless productions?

- a) $S \rightarrow A$
- b) $A \rightarrow aA$
- c) $A \rightarrow e$
- d) $B \rightarrow Ba$

6- Given grammar G:

$S \rightarrow aS \mid A \mid C$

$A \rightarrow a$

$B \rightarrow aa$

$C \rightarrow aCb$

Find the set of variables that can produce strings only with the set of terminals.

- a) $\{C\}$
- b) $\{A, B\}$
- c) $\{A, B, S\}$
- d) None of the mentioned